

Customer Churn Prediction

Using Machine Learning on E-Commerce Behavioral Data

Data Science - Final Project Report

Dataset: E-Commerce Customer Behavior and Sales Analysis

GitHub: [ChurnCustomer](#)

By: Itamar Babai, Rei Shaul, Raz Oununu

June 2026

Abstract

This report presents the design, implementation, and comparative evaluation of machine learning models for customer churn prediction in an e-commerce context. Using a dataset of 5,630 customer records with 20 behavioral and demographic features, we benchmarked five algorithms - Dummy Classifier, Logistic Regression, Random Forest, XGBoost, and CatBoost - against standard classification metrics. The study prioritizes Recall over Precision, reflecting the higher business cost of missing a churning customer. Our best - performing model, CatBoost with balanced class weights and a 30% probability threshold, achieves a Recall of 97.5% and ROC AUC of 98.15%, missing only 7 out of 284 churning customers in the test set.

1. Introduction

Customer churn - the decision by an existing customer to stop using a service - is one of the most costly challenges in e-commerce. Acquiring a new customer is substantially more expensive than retaining an existing one. Early identification of at-risk customers allows companies to deploy targeted interventions such as personalized offers, loyalty benefits, or proactive service improvements.

This project develops and compares machine learning models for binary classification of customer churn. The target variable distinguishes customers who churned (label 1) from those who remained (label 0). Because missing a churning customer carries a higher business cost than a false alarm, model selection prioritizes Recall while maintaining acceptable overall accuracy.

Five algorithm families were benchmarked: Dummy Classifier (baseline), Logistic Regression, Random Forest, XGBoost, and CatBoost. Each was evaluated on Accuracy, Balanced Accuracy, Precision, Recall, F1-Score, and ROC AUC.

2. Dataset & Working Assumptions

2.1 Dataset Overview

5,630	20	16.8%	70 / 30
Customer Records	Features	Churn Rate	Train / Test Split

The study uses the E-Commerce Customer Behavior and Sales Analysis dataset published on Kaggle. It contains 5,630 unique customer records and 20 columns covering demographics, app usage, purchase history, satisfaction scores, and complaint data. The target column Churn encodes customer attrition: 1 = churned, 0 = retained.

2.2 Feature Set

Numerical Features

Feature	Description	Range
Tenure	Customer tenure (months)	0–61
WarehouseToHome	Distance from warehouse to home (km)	5–127
HourSpendOnApp	Average monthly hours on app/site	0–5
NumberOfDeviceRegistered	Registered devices in account	1–6
SatisfactionScore	Customer satisfaction rating	1–5
NumberOfAddress	Saved addresses in account	1–21

Complain	Complaint lodged in the past year (0/1)	0 or 1
OrderAmountHikeFromlastYear	% order value increase vs. prior year	11–26
CouponUsed	Coupons redeemed last month	0–16
OrderCount	Orders placed last month	1–16
DaySinceLastOrder	Days since last order	0–46
CashbackAmount	Average cashback amount (₹)	0–325
CityTier	City tier (1=large, 3=small)	1, 2, 3

Categorical Features

Feature	Description	Possible Values
PreferredLoginDevice	Preferred login device	Mobile Phone, Computer
PreferredPaymentMode	Preferred payment method	Debit Card, UPI, Credit Card, COD, E-wallet
Gender	Customer gender	Male, Female
PreferedOrderCat	Preferred product category	Laptop & Accessory, Mobile, Fashion, Grocery, Others
MaritalStatus	Marital status	Single, Married, Divorced

2.3 Working Assumptions

Pre-modeling Assumptions

- CustomerID is a technical identifier only and was removed before training to prevent overfitting.
- The Churn column provides reliable ground-truth labeling.
- The dataset reflects a normal operating period, without confounding external events.
- Meaningful statistical relationships exist between the features and churn behavior.

Preprocessing Assumptions

- Missing values are handled by median imputation (numerical) and mode imputation (categorical), rather than row deletion.
- Categorical features are One-Hot Encoded, assuming no ordinal relationship between categories (Not at CatBoost).

Data Splitting Strategy

- A stratified 70/30 train-test split preserves the original churn distribution across both subsets.
- The test set remains completely unseen during model training and hyperparameter tuning.

Class Imbalance Handling

- The churn class represents 16.8% of observations and is treated as the minority class.
- Class-weight balancing is applied when appropriate to reduce majority-class bias and improve churn detection.

3. First Experiment: The 'Big Mistake'

Prior to the main study, an exploratory pipeline was built and validated using a synthetic dataset titled 'Returning Customers' from Kaggle. Four models - Dummy model ,Logistic Regression, Random Forest, and XGBoost - were trained on a 70/30 split.

All models converged to a ROC AUC of approximately 0.50, indistinguishable from the Dummy baseline. Post-hoc analysis revealed that the target variable had been generated by a fully random stochastic process, uncorrelated with any feature. There was zero mutual information between X and y.

Model	Accuracy	Precision	Recall	F1	ROC AUC
Dummy (baseline)	50%	50%	50%	50%	50%
Logistic Regression	50.01%	49.9%	51.2%	50.5%	50%
Decision Tree	50%	49.6%	49.9%	49.8%	50%
Random Forest	51%	50.04%	50.09%	50.7%	50.09%
XGBoost	49.9%	49.1%	49.3%	49.2%	49.8%

This finding served as a critical validation gate: it demonstrated empirically that model complexity or hyperparameter tuning cannot compensate for absent data quality. The 'Returning Customers' dataset was formally disqualified, and the research pipeline was migrated entirely to the E-Commerce Customer Churn dataset described above.

4. Methodology

4.1 Data Preprocessing Pipeline

- Excel source file read with Pandas, column names stripped of whitespace.
- CustomerID removed (non-predictive identifier).
- Missing values: median imputation for numerical, mode imputation for categorical.
- One-Hot Encoding applied to all 5 categorical features.
- Stratified 70/30 train-test split with random shuffle (shuffle=True, stratify=y).

4.2 Evaluation Metrics

Models are evaluated using Accuracy, Balanced Accuracy, Precision, Recall, F1-Score and ROC AUC. Given the business cost asymmetry - a missed churner is far more costly than a false positive -

Recall on the churn class is the primary selection criterion, with ROC AUC as a model-quality sanity check.

5. Model Experiments

5.1 Dummy Classifier - Baseline

The Dummy Classifier makes predictions using only the class distribution, ignoring all features. Three strategies were tested:

most_frequent - Always predicts 0 (retained) - the majority class. Achieves 83.2% accuracy purely because 83.2% of customers don't churn. Recall on churners = 0. Completely useless from a business perspective.

stratified - Predicts randomly according to the class distribution in the data - 83.2% of the time predicts 0, 16.8% predicts 1. ROC AUC of ~0.50, meaning pure random guessing.

uniform - Predicts 0 or 1 with equal probability (50/50). Also achieves ROC AUC of ~0.50.

Notably, most_frequent achieves 83.2% accuracy simply because 83.2% of customers did not churn - illustrating why raw accuracy is a misleading metric on imbalanced datasets. Its ROC AUC of 0.50 confirms pure random guessing.

Metric	most frequent	stratified	uniform
Accuracy	83.2%	71.7%	50.8%
Balanced Accuracy	50.0%	50.3%	51.2%
Precision (churners)	0.0%	17.4%	17.5%
Recall (churners)	0.0%	18.0%	51.8%
F1 (churners)	0.0%	17.7%	26.2%
ROC AUC	50%	50%	50%
Missed churners	284 / 284	233 / 284	~137 / 284

Lower Bounds Any real model must beat:	ROC AUC > 50%	Recall > 51.8%	F1 > 26.2%
--	-------------------------	--------------------------	----------------------

5.2 Logistic Regression

Algorithm Overview

Logistic Regression is a linear probabilistic classification model. It estimates the probability that a customer will churn using a weighted combination of input features, then converts that probability into a class prediction using a decision threshold. Because the model is simple it serves as a strong baseline before moving to non-linear tree-based models.

Experiment 1 - Baseline Logistic Regression, Threshold 0.5

A stratified 70/30 train-test split was applied. The baseline model used the standard preprocessing pipeline with median imputation, standard scaling for numerical variables, one-hot encoding for categorical variables, and no class-weight adjustment or feature engineering.

Logistic Regression - Experiment 1 - Threshold 0.50

	Predicted: Stayed (0)	Predicted: Churned (1)	
Actual: Stayed (0)	✓ 1,362 True Negative	✗ 43 False Positive	1405 Actual Stayed
Actual: Churned (1)	✗ 140 False Negative (Missed churners)	✓ 144 True Positive	284 Actual Churned
	1502	187	1689

Model	Accuracy	Balanced Accuracy	Recall	Precision	F1	ROC AUC
LR baseline @ 0.50	89.2%	73.82%	50.7%	77.0%	61.2%	88.4%

The baseline model achieves a ROC AUC of 88.4%, confirming that the dataset contains meaningful signal even for a simple linear classifier. However, Recall is only 50.7%, meaning 140 of 284 churners in the test set were missed.

Experiment 2 - Balanced Class Weights + Feature Engineering + Threshold Tuning

Experiment 2 added three improvements to the baseline Logistic Regression model. First, `class_weight='balanced'` was used to handle the imbalanced target by giving more importance to churned customers during training. Second, four engineered features were added to better represent customer behavior, including complaints (Complain), low satisfaction (satisfactionScore), cashback per order (ChashBackAmount), and recency of the last order (DaySinceLastOrder). Third,

both the regularization parameter C and the decision threshold were tuned on the training set only. The final model was then evaluated once on the holdout test set to avoid data leakage.

Threshold	Accuracy	Balanced Accuracy	Recall	Precision	F1	ROC AUC
0.50 (balanced)	79.5%	80.52%	82.0%	44.1%	57.4%	80.5%
0.7365 ★ (balanced)	87.3%	77.46%	62.7%	62.0%	62.4%	77.5%

The threshold scan shows the central trade-off: threshold 0.50 **maximizes Recall (82.0%)** and catches 233 of 284 churners, but Precision drops to 44.1%. The tuned threshold 0.7365 provides a more balanced operating point, with Precision and Recall both near 62%.

Logistic Regression - Selected Balanced Point - Threshold 0.7365

	Predicted: Stayed (0)	Predicted: Churned (1)	
Actual: Stayed (0)	✓ 1,296 True Negative	✗ 109 False Positive	1405 Actual Stayed
Actual: Churned (1)	✗ 106 False Negative (Missed churners)	✓ 178 True Positive	284 Actual Churned
	1402	287	1689

Logistic Regression establishes a useful interpretable benchmark: the model can rank churn risk reasonably well, but its linear structure limits its ability to capture richer interactions between complaints, tenure, cashback, device count, and purchase behavior.

5.3 Random Forest

Algorithm Overview

Random Forest is an ensemble algorithm based on Bootstrap Aggregating (Bagging). It trains 100 independent decision trees on random row-and-column subsets of the data, then aggregates predictions by majority vote. A maximum depth of 12 levels constrains each tree, preventing overfitting while preserving sufficient capacity to model non-linear feature interactions.

Experiment 1 - Default Threshold (0.50)

A stratified 70/30 split was applied. The model was evaluated at the standard 0.50 probability threshold with no class-weight adjustment.

Random Forest - Experiment 1 - Threshold 0.50

	Predicted: Stayed (0)	Predicted: Churned (1)	
Actual: Stayed (0)	✓ 1,378 True Negative	✗ 27 False Positive	1405 Actual Stayed
Actual: Churned (1)	✗ 66 False Negative (Missed churners)	✓ 218 True Positive	284 Actual Churned
	1444	245	1689

Model	Accuracy	Balanced Accuracy	Recall	Precision	F1	ROC AUC
RF @ 0.50	94.5%	87.4%	76.8%	89.0%	82.4%	97.9%

The model achieves excellent overall accuracy and a **very high ROC AUC of 97.9%**, indicating strong rank-ordering ability. **However, Recall of 76.8%** means 66 churning customers were missed - the primary weakness from a business perspective.

Experiment 2 - Probability Threshold Scan (0.20–0.60)

A dynamic threshold scan was performed on the existing model's probability outputs (no retraining) to find the optimal operating point for the business objective.

Threshold	Accuracy	Balanced Accuracy	Recall	Precision	F1	Missed FN
0.60	93.8%	83%	66.9%	94.5%	78.4%	94
0.55	94.1%	85.1%	71.5%	91.9%	80.4%	81
0.50 (default)	94.5%	87.4%	76.8%	89.0%	82.4%	66
0.45 (best F1)	94.6%	89.4%	81.7%	85.3%	83.5%	52

0.40	93.8%	90.9%	86.6%	78.9%	82.6%	38
0.35 ★ (selected)	93.4%	92.2%	90.5%	75.4%	82.2%	27
0.30	92.4%	93.2%	94.4%	70.5%	80.7%	16
0.25	90.5%	92.7%	96.1%	64.5%	77.2%	11
0.20	86.4%	90.8%	97.5%	55.5%	70.8%	7

The scan reveals a clear trade-off: lowering the **threshold increases Recall** (fewer missed churners) **but reduces Precision** (more false alarms). Two notable operating points emerge:

- Threshold 0.45 - Maximum F1 (83.5%): best mathematical balance between Precision and Recall.
- **Threshold 0.35 - Selected operating point: Recall rises to 90.5%, catching 257 of 284 churners, while overall accuracy remains high at 93.4%.**

Random Forest - Selected Operating Point - Threshold 0.35

	Predicted: Stayed (0)	Predicted: Churned (1)	
Actual: Stayed (0)	✓ 1,321 True Negative	✗ 84 False Positive	1405 Actual Stayed
Actual: Churned (1)	✗ 27 False Negative (Missed churners)	✓ 257 True Positive	284 Actual Churned
	1328	341	1689

The 0.35 threshold was selected for the Random Forest as the production configuration, reducing missed churners from 66 to 27 (a 59% reduction) at the cost of a minor accuracy decrease of ~1% (93.4% instead of 94.6%).

5.4 XGBoost

Algorithm Overview

XGBoost is a gradient boosting algorithm that builds decision trees sequentially, where each new tree corrects the errors made by the previous ensemble. It was selected because it is highly effective on tabular datasets, captures non-linear relationships between customer behavior variables, and often provides stronger predictive performance than simpler linear models. In this project, class imbalance was handled using `scale_pos_weight`, computed from the training set only.

Experiment 1 - Baseline XGBoost + `scale_pos_weight`

The first XGBoost experiment used the original feature set, the same stratified 70/30 split, the shared preprocessing pipeline, and `scale_pos_weight=4.94` to compensate for the 83/17 class imbalance. The model was evaluated at the default 0.50 probability threshold.

XGBoost - Experiment 1 - Threshold 0.50

	Predicted: Stayed (0)	Predicted: Churned (1)	
Actual: Stayed (0)	✓ 1,265 True Negative	✗ 140 False Positive	1405 Actual Stayed
Actual: Churned (1)	✗ 32 False Negative (Missed churners)	✓ 252 True Positive	284 Actual Churned
	1297	392	1689

Model	Accuracy	Balanced Accuracy	Recall	Precision	F1	ROC AUC
XGB Exp 1 @ 0.50	89.8%	89.38%	88.7%	64.3%	74.6%	94.8%

XGBoost reaching ROC AUC 94.8% and Recall 88.7%. It identifies 252 of 284 churners, missing only 32 churners at the default threshold.

Experiment 2 - Feature Engineering + Threshold Tuning

The second experiment reused the four engineered features from Logistic Regression and tuned the decision threshold on the training set. No test-set information was used during threshold selection; the final thresholds were evaluated once on the holdout test set.

Threshold	Accuracy	Balanced Accuracy	Recall	Precision	F1	ROC AUC
0.50 (Exp 2)	89.5%	88.89%	88.0%	63.5%	73.8%	94.86%
0.7178 ★ (Exp 2)	91.7%	84.17%	72.9%	76.4%	74.6%	94.86%

Feature engineering produced **almost no change in ROC AUC (94.86%)**, suggesting that **XGBoost already captures the relevant feature interactions**. Threshold tuning mainly changes the operating point: threshold 0.50 favors Recall, while threshold 0.7178 increases Precision.

XGBoost - Selected Balanced Point - Threshold 0.7178

	Predicted: Stayed (0)	Predicted: Churned (1)	
Actual: Stayed (0)	✓ 1,341 True Negative	✗ 64 False Positive	1405 Actual Stayed
Actual: Churned (1)	✗ 77 False Negative (Missed churners)	✓ 207 True Positive	284 Actual Churned
	1418	271	1689

At threshold 0.7178, XGBoost reaches 91.7% accuracy, 76.4% precision, and 72.9% recall. This configuration is appropriate when retention actions are relatively costly and the business prefers fewer false-positive alerts.

5.5 CatBoost

Algorithm Overview

- **Algorithm Overview** - CatBoost is a gradient boosting model based on decision trees. It builds many trees one after another. Each new tree tries to fix the mistakes made by the previous trees.
- **Why CatBoost Was Selected** - CatBoost was selected because it works very well with tabular data and categorical features. Unlike models that require manual One-Hot Encoding, CatBoost can handle categorical columns directly.
- **Handling Categorical Features** - CatBoost converts each category into a numerical value based on the target statistics of that category. For example, if customers with a certain preferred order category tend to churn more often, CatBoost can learn this pattern and use it during training.
- **Reducing Target Leakage and Overfitting** - Another advantage of CatBoost is its Ordered Boosting method. CatBoost does not allow each row to use its own target value when creating these category-based statistics. Instead, it uses random ordered permutations of the data, so each row is encoded only using information from previous rows in the order. This helps prevent target leakage and reduces overfitting, especially when the dataset is not very large.
- **Strong Default Performance** - CatBoost also often gives strong results with default parameters, meaning it usually requires less manual tuning than XGBoost. In this project, this was useful because the first CatBoost experiment already achieved a very high ROC AUC score of about 0.98.

Experiment 1 - Default Parameters, Threshold 0.50

CatBoost was trained with default parameters and no class-weight adjustment, using the same 70/30 stratified split.

CatBoost - Experiment 1 - Threshold 0.50

	Predicted: Stayed (0)	Predicted: Churned (1)	
Actual: Stayed (0)	✓ 1,380 True Negative	✗ 25 False Positive	1405 Actual Stayed
Actual: Churned (1)	✗ 70 False Negative (Missed churners)	✓ 214 True Positive	284 Actual Churned
	1450	239	1689

Metric	Accuracy	Balanced Accuracy	Recall	Precision	F1	ROC AUC
CatBoost Exp 1	94.4%	86.79%	75.4%	89.5%	81.8%	98.0%

Experiment 1 confirms strong baseline performance with ROC AUC of 0.980 and Precision of 89.5%. The main weakness is Recall of 75.4% - 70 churning customers were missed.

Experiment 2 - Balanced Class Weights + Threshold Scan

Two changes were introduced relative to Experiment 1: (1) `auto_class_weights='Balanced'` instructs CatBoost to automatically assign higher loss weight to the minority churn class during training; (2) three probability thresholds (0.30, 0.40, 0.50) were evaluated on the test set.

CatBoost Exp 2 - Threshold 0.50

	Predicted: Stayed (0)	Predicted: Churned (1)	
Actual: Stayed (0)	✓ 1,339 True Negative	✗ 66 False Positive	1405 Actual Stayed
Actual: Churned (1)	✗ 22 False Negative (Missed churners)	✓ 262 True Positive	284 Actual Churned
	1361	328	1689

CatBoost Exp 2 - Threshold 0.40

	Predicted: Stayed (0)	Predicted: Churned (1)	
Actual: Stayed (0)	✓ 1,308 True Negative	✗ 97 False Positive	1405 Actual Stayed
Actual: Churned (1)	✗ 16 False Negative (Missed churners)	✓ 268 True Positive	284 Actual Churned
	1324	365	1689

CatBoost Exp 2 - Threshold 0.30

	Predicted: Stayed (0)	Predicted: Churned (1)	
Actual: Stayed (0)	✓ 1,270 True Negative	✗ 135 False Positive	1405 Actual Stayed
Actual: Churned (1)	✗ 7 False Negative (Missed churners)	✓ 277 True Positive	284 Actual Churned
	1277	412	1689

Metric	Exp 1 (t=0.50)	Exp 2 (t=0.50)	Exp 2 (t=0.40)	Exp 2 (t=0.30) ♦
--------	----------------	----------------	----------------	------------------

Accuracy	94.4%	94.8%	93.3%	91.6%
Balanced Accuracy	86.79%	93.78%	93.73%	93.96%
ROC AUC	97.96%	98.15%	98.15%	98.15%
Recall (churners)	75.4%	92.3%	94.4%	97.5%
Precision (churners)	89.5%	79.9%	73.4%	67.2%
F1 (churners)	81.8%	85.6%	82.6%	79.6%
Missed churners (FN)	70 / 284	22 / 284	16 / 284	7 / 284

Adding balanced class weights produced a dramatic improvement in Recall - from 75.4% to 92.3% at threshold 0.50 with only a moderate Precision decrease. ROC AUC improved slightly to 0.9815. Further lowering the threshold to 0.30 pushes Recall to 97.5%, missing only 7 churners, at the cost of 135 false positives.

6. Cross-Model Comparison

The table below consolidates the performance of all model configurations evaluated in this study, spanning from the lower bound (Dummy) to the best-performing CatBoost configuration.

Model	Threshold	Accuracy	Balanced Accuracy	Recall	Precision	F1	ROC AUC	Missed FN
Dummy (stratified)	—	71.7%	51.2%	18.0%	17.4%	17.7%	50%	233 / 284
Random Forest 1	50%	94.5%	86.4%	76.8%	89.0%	82.4%	97.9%	66 / 284
Random Forest 2	0.35 ★	93.4%	92.2%	90.5%	75.4%	82.2%	97.9%	27 / 284
CatBoost Exp 1	50%	94.4%	86.79%	75.4%	89.5%	81.8%	98%	70 / 284
CatBoost Exp 2	50%	94.8%	93.78%	92.3%	79.9%	85.6%	98.2%	22 / 284
CatBoost Exp 2	40%	93.3%	93.73%	94.4%	73.4%	82.6%	98.2%	16 / 284
CatBoost Exp 2	0.30 ✦	91.6%	93.96%	97.5%	67.2%	79.6%	98.2%	7 / 284

★ Selected Random Forest operating point

✦ Best overall model (recommended for deployment)

6.1 Recommended Model and Configuration

Recommended: CatBoost Experiment 2, Threshold = 0.30

Recall: 97.5% | ROC AUC: 98.15% | Missed churners: 7 / 284

6.2 Decision Rationale

- This recommendation as mentioned before based on the following assumption:
We assume that the business prefers to invest resources on customers that are supposed to leave, even though they may not actually churn (high recall and may be low precision).
On the other hand, if the business prefers to invest its resources only in customers who are highly likely to churn and invest less in the customers who may churn, we recommend using a different model with higher precision (with adjustment).
- After adding balanced class weights and lowering the threshold to 0.30, Recall improved to 97.5%. This reduced the number of missed churners to only 7 out of 284. Although the threshold is aggressive, the model still keeps a strong overall accuracy of 91.6%.
- ROC AUC of 0.9815 (vs. Dummy's 0.50) confirms the model is genuinely learning business-relevant patterns, not exploiting class imbalance.

7. Feature Importance Analysis

In the Random Forest model, feature importance was computed using the Mean Decrease in Impurity (MDI) metric, which measures the cumulative reduction in Gini Impurity across all decision trees in the forest. To illustrate: suppose one tree first splits on Tenure. Before the split, the node contains a mixture of churned and retained customers. After the split, low-tenure customers concentrate in one branch and high-tenure customers in the other, making each branch more homogeneous with respect to the target variable. This reduction in node impurity is attributed to Tenure. The process repeats at every split across all trees in the forest. At the end of training, the algorithm sums all impurity reductions caused by each feature and normalizes them so that the total importance scores sum to 100%. A feature that receives a high score is therefore one that consistently produced high-quality splits separating churners from retained customers.

Since Random Forest cannot handle categorical variables directly, One-Hot Encoding was applied during preprocessing to all five categorical features. As a result, a feature such as MaritalStatus was decomposed into separate binary columns (e.g., Married, Single, and Divorced), and the model computed an individual importance score for each. For comparison with CatBoost, the aggregated importance of each categorical feature was obtained by summing the importance scores of all its derived binary columns - so the final importance of MaritalStatus, for example, equals the sum of MaritalStatus_Single, MaritalStatus_Married, and MaritalStatus_Divorced.

In the CatBoost model, feature importance was computed using the PredictionValuesChange metric. Unlike Random Forest, this metric does not assess the quality of individual splits but rather the effect of each feature on the model's final prediction. To illustrate: suppose the model estimates a churn probability of 20% for a given customer. If varying the value of Tenure causes that estimate to shift to 55% or drop to 8%, then Tenure has produced a substantial change in the predicted output and will receive a high importance score. Conversely, a feature whose variation barely affects the predicted probability will receive a low score. The algorithm computes each feature's influence on predictions across all boosting trees and aggregates its cumulative contribution to the model output.

An additional advantage of CatBoost is its ability to handle categorical variables natively, without One-Hot Encoding. Rather than decomposing PreferredOrderCat into multiple binary columns, the model receives the original categorical values and computes a single importance score for the feature as a whole. Importance scores are therefore produced directly for all 18 original features in the dataset, with no post-training aggregation required. For cross-model comparison, the importance scores of both models were normalized so that each sums to 100%, enabling direct comparison of each feature's relative contribution despite fundamental differences in how the two models compute importance and learn from data.

Feature	Importance by CatBoost	Importance by RandomForest
Tenure	17.88%	26.92%
SatisfactionScore	8.73%	4.14%
NumberOfAddress	8.19%	4.73%
PreferredOrderCat	8.01%	6.12% (Laptop Accessory & Mobile Phone & Mobile & Fashion & Grocery & others)
WarehouseToHome	7.58%	5.78%
Complain	7.07%	6.98%
CashbackAmount	5.91%	8.71%
OrderAmountHikeFromlastYear	5.43%	4.33%
DaySinceLastOrder	4.93%	5.97%
PreferredPaymentMode	4.77%	5.13% (Credit Card & Debit Card & E wallet & COD & UPI & CC & Cash on Delivery)
CityTier	4.11%	2.40%
MaritalStatus	4.07%	4.70% (Single & Married & Divorced)
NumberOfDeviceRegistered	3.13%	2.82%
PreferredLoginDevice	3.09%	2.75% (Mobile Phone & Computer & Phone)
Gender	2.45%	2.18% (Female & Male)
OrderCount	2.32%	2.73%
CouponUsed	1.34%	2.21%

HourSpendOnApp	0.97%	1.31%
----------------	-------	-------

For each model, percentages add up to approximately 100% (Random Forest approximates due to rounding to 2 digits after the decimal point)

The feature importance analysis across both models confirms that **Tenure** is by far the most influential predictor of customer churn, ranking first in both Random Forest and CatBoost by a substantial margin over all other features. Features such as **Complain**, **DaySinceLastOrder**, and **WarehouseToHome** ranked consistently among the top predictors in both models, reinforcing the conclusion that service experience, activity frequency, and logistical distance are central drivers of churn behavior.

On the other end of the spectrum, **HourSpendOnApp** and **Gender** received low importance scores in both models - a somewhat surprising finding, given the expectation that digital engagement and demographic characteristics would exert a more meaningful influence on consumer behavior.

Regarding differences between the models, **CashbackAmount** ranked notably higher in Random Forest (2nd, 8.71%) than in CatBoost (7th, 5.91%), while **SatisfactionScore** showed the opposite pattern - ranking 2nd in CatBoost (8.73%) but considerably lower in Random Forest (4.14%). These divergences reflect the fundamental differences in how each model computes importance: Random Forest rewards features that produce clean Gini splits across numerical values, while CatBoost measures direct impact on predicted output, which may weight the same features differently depending on their interaction with other variables in the boosting sequence.

8. Challenges Encountered

- Finding a reliable and authentic dataset: Initial attempts used a synthetic dataset that turned out to have randomly generated labels - discovered only after building the full modeling pipeline (see Section 3).
- Finding a project topic that is both interesting and technically meaningful, with enough features to support real machine learning analysis.
- Class imbalance (83.2% retained vs. 16.8% churned): Standard accuracy metrics were misleading. The solution - combining `class_weight='balanced'` with threshold tuning - was iteratively discovered through experimentation.
- Data split strategy: Multiple strategies were explored including 5-fold cross-validation and shuffle-only splits before settling on stratified holdout with shuffle, which balances chronological bias prevention with reproducibility.
- transforming findings into a clear, academic narrative while preserving all quantitative detail.

9. Conclusions & Future Work

- This project demonstrates that machine learning can reliably identify at-risk customers in an e-commerce setting. Starting from a failed pilot on a synthetic dataset, the research established a validated methodology and delivered a production-ready model with near-ceiling churn detection performance.
- The key methodological insight is that threshold calibration - rather than model selection alone - is the most actionable lever for aligning ML outputs with business cost structures. Reducing the CatBoost classification threshold from 0.50 to 0.30 improved Recall by 22

percentage points at the cost of 2.8 points of overall accuracy, a trade-off strongly favorable in the churn context.

- Threshold tuning and class weight adjustment were highly impactful - lowering the classification threshold and applying `auto_class_weights="Balanced"` significantly improved Recall on churners, reducing missed churners to just 7. These techniques proved to be among the most effective tools in addressing class imbalance without modifying the data itself.
- Tenure is the most influential feature - the number of consecutive months a customer has been active is the strongest predictor of churn. Customers with low tenure (0-1 months) churn at a rate exceeding ~50%, while long-term customers (22+ months) show a churn rate close to 0%.
- Both linear and non-linear relationships exist between features and the target - unlike synthetic datasets where relationships are purely random, this dataset contains genuine structure. Features such as Tenure, Complain, and CashbackAmount exhibit meaningful patterns that the model was able to learn and exploit.
- Both tree-based models performed well and identified similar key churn indicators. However, CatBoost used the features in a more balanced way. Random Forest gave almost 27% of its total importance to Tenure alone, while CatBoost spread importance across several behavioral and satisfaction-related features. This suggests that CatBoost captures more complex customer patterns without losing interpretability.
- Native categorical feature handling proved more effective than traditional One-Hot Encoding. By leveraging CatBoost's built-in Ordered Target Statistics, the model was able to extract predictive information directly from categorical variables without expanding them into dozens of binary features. This approach preserved the original structure of the data, reduced feature sparsity, and contributed to improved churn detection performance. The results suggest that native categorical processing is particularly advantageous for e-commerce datasets containing multiple customer and behavioral categories.

Planned Extensions:

- Feature Engineering - Create new features from existing ones - for example, a ratio between OrderCount and Tenure to capture purchase frequency, or a binary flag indicating whether a customer filed a complaint in their first month. These derived features may expose patterns that the raw features alone cannot capture.
- Hyperparameter Tuning with Optuna - Systematically search for the optimal combination of CatBoost parameters (iterations, depth, learning rate, l2 regularization) using Bayesian optimization. This could further improve ROC AUC and F1 beyond what the baseline configuration achieved.
- New Data - Evaluate the model on external or more recent data to test generalization beyond the current dataset.
- Budget-Constrained Model Adaptation - In real-world scenarios, businesses have a limited retention budget and can only contact a fixed number of customers per month. Rather than applying a static threshold, the model can adopt a **dynamic threshold** driven by the available budget - ranking customers by predicted churn probability and selecting the top-K most at-risk. This transforms the model from a statistical tool into a **business-driven decision system**.